



Migrating an on-premises SAP ASE database to Amazon RDS for SQL Server

AWS Prescriptive Guidance



AWS Prescriptive Guidance: Migrating an on-premises SAP ASE database to Amazon RDS for SQL Server

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction **1**

Objectives 1

Intended audience 1

Migration **2**

Migration phases 2

Phase 1: Prepare 2

Phase 2: Plan 3

Phase 3: Migrate 4

Phase 4: Operate and optimize 10

Resources **11**

AWS documentation 11

AWS services 11

Blog posts 11

Sybase ASE and SQL Server 12

Document history **13**

Migrating an on-premises SAP ASE database to Amazon RDS for SQL Server

Amit Kumar, Jitendra Kumar, and Nishad Mankar, Amazon Web Services

This guide describes options for migrating an SAP ASE database to [Amazon Relational Database Service \(Amazon RDS\) for SQL Server](#). Amazon RDS for SQL Server is a common migration choice for SAP ASE workloads because it supports Transact-SQL (T-SQL) commands and features. It also supports Tabular Data Stream (TDS), which is the native client-server communication protocol for SQL Server.

Organizations often migrate from [SAP Adaptive Server Enterprise \(SAP ASE\)](#) to Amazon RDS for SQL Server to reduce licensing costs and automate database management using AWS services. Amazon RDS automates administration tasks like backups, software patching, and high-availability features while maintaining performance and security.

Objectives

This guide describes how to migrate applications that run on SAP ASE databases to AWS. Learn about migration approaches, implementation phases, and specific steps for replatforming and refactoring your applications. Select a migration strategy that aligns with your business requirements and technical environment to help reduce downtime and optimize costs.

Intended audience

This guide is for database administrators, database engineers, and migration engineers who intend to migrate an SAP ASE database to the AWS Cloud. This guide assumes that you have knowledge of the following services and concepts:

- Amazon RDS for SQL Server and related database technologies
- AWS Database Migration Service (AWS DMS) features, limitations, and security requirements
- Using SAP ASE as a source for AWS DMS, including limitations with SQL Server as a target
- Network setup for AWS DMS replication
- Schema conversion tools for SAP ASE and SQL Server

Migration

When choosing a migration strategy, consider your business requirements, resources, time, and finances.

Migration phases

To provide a single framework for migrating various workloads—such as online transaction processing (OLTP) and online analytic processing (OLAP)—this guide is organized into four migration phases that support most applications. The following are descriptions of common migration phases.

- [Phase 1: Prepare](#) – Consider the database workloads to determine if your database migration is homogeneous or heterogeneous.
- [Phase 2: Plan](#) – Evaluate the data to determine a suitable migration strategy, such as rehost, replatform, repurchase, refactor, retain, or retire.
- [Phase 3: Migrate](#) – After you choose a database migration category and strategy, create the target database and migrate your source database using data validation and replication.
- [Phase 4: Operate and optimize](#) – Maximize the benefits of hosting applications on AWS, which include addressing costs, performance, security, and application resilience.

Phase 1: Prepare

Assess the source SAP ASE database architecture to understand its dependencies and current state.

Assess the source database

To assess the source database, such as objects, users, and permissions, you can use third-party tools. For more information, see [Preparation and assessment for migrating from SAP ASE to Amazon Aurora MySQL](#).

- Analyze the code conversion by listing all of the application dependencies and interfaces.
- Identify SAP application dependencies that are only supported in SAP. You may need to find an alternative method to work with Microsoft SQL.
- Identify communication protocols.

- Identify replication requirements and high availability/disaster recovery compliance.
- Identify security compliance and encryption protocols.

You can use any third-party migration tool for converting the schema code, such as SAP ASE database or Amazon RDS for SQL Server. You must convert some code manually, such as non-ANSI SQL, because SQL Server may not support it. After you convert the code, conduct unit and functional testing to validate database objects. For more information, see [Migrate your SAP ASE \(Sybase ASE\) database to Amazon RDS for SQL Server](#).

Evaluate security concerns

SQL Server offers various security options that are based on enterprise compliance requirements and unique use cases. For more information, see the following documents about the security of SQL Server:

- [Authenticate Microsoft SQL Server on Amazon EC2 using AWS Directory Service](#)
- [Enable transparent data encryption in Amazon RDS for SQL Server](#)

Consult AWS Partners

To reduce migration times, consider consulting with [AWS Database Migration Service Partners](#). AWS Partners have the necessary expertise to help with both homogeneous and heterogeneous migrations between different database platforms, such as SAP ASE to Amazon Aurora MySQL or Microsoft SQL Server to MySQL.

Based on your requirements and preferences, partners can provide tools and advice for how to handle migrations or troubleshoot migration aspects. Ensure that you follow AWS best practices to host your database in the AWS Cloud. For more information, see [Best practices for Amazon RDS](#).

Phase 2: Plan

Based on your workload and application, choose either [RDS custom for SQL Server](#) or [RDS for Microsoft SQL Server](#). For most database migrations, you can rehost, replatform, or refactor. The guiding principle should be how to maximize a migration's benefits.

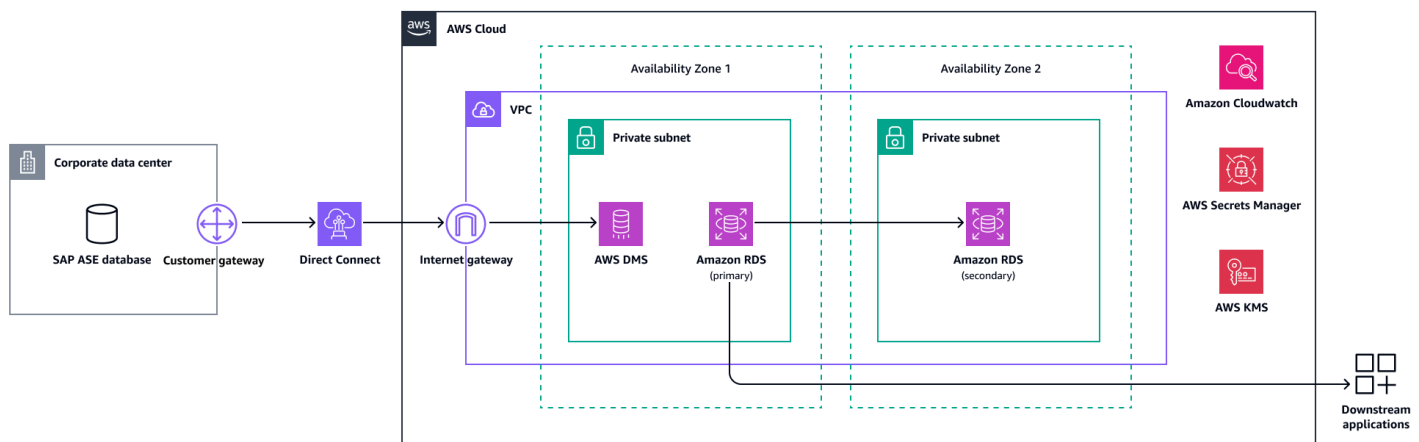
Choosing to refactor your application and migrate to a cloud-native database such as Aurora can enhance your database application. Depending on your workload complexity, however, refactoring a database can be time-consuming and resource-intensive.

Migration option for Amazon RDS for SQL Server

Consider whether your source database is already set up to synchronize primary and secondary database servers, and then decide how you want to synchronize your servers on AWS. The following are three approaches:

- Multiple Availability Zones
- Region replication
- [Cross-Region read replicas](#)

In the following architecture diagram, AWS Direct Connect establishes a dedicated, private network connection between the on-premises data center and Amazon VPC. The target database environment has a primary Amazon RDS for SQL Server deployed in one Availability Zone, with a read replica in a second Availability Zone for high availability and disaster recovery. AWS Database Migration Service (AWS DMS) manages the migration process to help minimize downtime.



For enhanced security, the target environment uses private subnets. AWS Secrets Manager handles user credentials, and AWS Key Management Service (AWS KMS) provides encryption at rest for sensitive data. The entire infrastructure is monitored by Amazon CloudWatch, which tracks performance metrics and enables proactive alerting. This design helps to optimize performance while addressing any issues that arise during the migration process.

Phase 3: Migrate

A primary task of database migrations is to complete the migration with minimal downtime. Because both databases should use the same programming language and communication

protocols, you may need to convert the code and schema for similar query syntaxes, procedures, and functions. When converting a schema, consider the following aspects:

- Modify the database connection as appropriate for the new engine.
- Fix any warnings and errors from the code conversion.
- Modify table mappings and code as appropriate for the converted schema.
- Identify and refactor any vendor-specific functionality that your application uses.

You can use any third-party migration tool for schema code conversion, such as SAP ASE database and Amazon RDS for SQL Server. You may need to convert some code manually because non-ANSI SQL is not supported in SQL Server.

After you convert the code, convert your application code or Dynamic SQL application, and then perform unit and functional testing. For more information, see [Testing Migrated Database Objects \(SybaseToSQL\)](#).

Converting the data

Transform raw data to make it more useful by cleaning, standardizing, verifying, and sorting it. In database migrations, extract, transform, and load (ETL) processes are used in the following ways:

- Inside the database
- With external scripts
- Using third-party tools

Examples of ETL tools include AWS Glue, Informatica, and Talend. For migrations from SAP ASE to SQL Server, some free tools can convert stored procedures and functions automatically.

Validating database objects

Validating your database helps prevent problems in subsequent migration stages. After code conversion, validate your database schema by comparing the following elements between SAP ASE and RDS SQL Server:

- Schemas
- Tables
- Views

- Functions
- Stored procedure indexes
- Triggers
- Constraints (for example, primary keys, foreign keys, checks, and defaults)

Check that each object migrated correctly. If you find differences, identify the reason for the failure. You may need to manually create missing objects in the target database or convert Transact-SQL code. For more information, see [Validate database objects after migrating from SAP ASE to Amazon RDS for SQL Server or Microsoft SQL Server](#).

Migrating data using AWS DMS

If you have multiple database users, your application may need to be migrated according to a schedule. Depending on the database size and migration window, such data migrations require knowledge of full loads and incremental loads. For this reason, AWS DMS can connect source and target databases to replicate database contents according to the following processes:

- Create a replication server.
- Create source and target endpoints that describe data store connections.
- Create one or more migration tasks to migrate data between source and target data stores.
- Ongoing replication from SAP ASE to SQL Server
- (Optional) Complete data migration from SAP ASE to SQL Server with change data capture

You may need to optimize AWS DMS for handling certain data types. For more information, see [Using an SAP ASE database as a source for AWS DMS](#).

Migrating offline data

You can use AWS Storage Gateway to integrate your SAP ASE database with Amazon Simple Storage Service (Amazon S3), which provides cost-effective, scalable, and secure storage for on-premises SAP ASE database backups. For more information, see [Integrate an SAP ASE database to Amazon S3 using AWS Storage Gateway](#).

Using third-party tools

Some applications serve as a single point of contact (SPOC) that interfaces with other applications. When migrating to an SQL Server database platform, these interconnections may be affected,

and database monitoring might require native or third-party tools that use server-specific communication protocols. It's important to assess whether these dependent applications and tools already support SQL Server or if they need modifications to function properly.

For packaged applications, consult vendors to determine if they support Amazon RDS for SQL Server. For custom applications, you may need to modify the code to ensure compatibility with the migrated database.

Monitoring the database

Regardless of your chosen migration path, Amazon CloudWatch plays a role in collecting metrics, such as CPU type, memory, and I/O functions. It's also capable of setting metrics thresholds and initiating actions when thresholds are triggered.

For example, you can create alarms for Amazon RDS cluster metrics, notifications, and actions to detect and shut down unused or underutilized reader instances. Setting alarms on metrics and events can help minimize downtime and business impacts. AWS services like Amazon S3, [Amazon RDS Performance Insights](#), Amazon CloudWatch, and AWS CloudTrail are already integrated with the RDS database platform, and we recommend them to monitor performance.

Validating the data

After the data migration from SAP ASE to Amazon RDS for SQL Server is complete, validate the data to ensure accuracy and consistency. Use the following SQL queries to generate metadata statements for each table in your database.

Step 1: Generate metadata statements and column lists

```
SELECT dt.schema_name, dt.table_name,
       STRING_AGG(dt.column_name, ',') AS column_name,
       STRING_AGG(dt.cname, ',') AS column_order
FROM (
  SELECT
    object_name(a.id) AS table_name,
    a.name colname,
    c.name col_type,
    a.isnullable,
    a.name AS cname,
    schema_name(b.uid) AS schema_name,
    CASE
      WHEN a.isnullable = 1 THEN
```

```

CASE
    WHEN c.name LIKE '%char%'
        THEN 'coalesce(ltrim(rtrim('+a.name+')), 'X') as '+a.name
    WHEN (c.name LIKE '%int%' OR c.name = 'numeric')
        THEN 'coalesce('+a.name+',0) as '+a.name
    WHEN c.name IN ('decimal','float','money')
        THEN 'coalesce('+a.name+',0.0) as '+a.name
    WHEN c.name LIKE 'datetime%'
        THEN 'coalesce(convert(nvarchar(30),'+a.name
+',112),'99991231')) as '+a.name
    ELSE a.name
END
WHEN c.name LIKE 'datetime%'
    THEN 'coalesce(convert(nvarchar(30),'+a.name+',112),'99991231')) as
'+a.name
    WHEN c.name LIKE '%char%'
        THEN 'coalesce(ltrim(rtrim('+a.name+')), 'X') as '+a.name
    ELSE a.name
END AS column_name
FROM syscolumns a
INNER JOIN sysobjects b
    ON a.id = b.id
    AND b.type = 'U'
INNER JOIN systypes c
    ON a.usertype = c.usertype
    AND a.xusertype = c.xusertype
    AND c.name != 'varbinary'
INNER JOIN (
    SELECT
        OBJECT_NAME(ic.OBJECT_ID) AS table_name,
        COL_NAME(ic.OBJECT_ID, ic.column_id) AS column_name
    FROM sys.indexes AS i
    INNER JOIN sys.index_columns AS ic
        ON i.OBJECT_ID = ic.OBJECT_ID
        AND i.index_id = ic.index_id
        AND i.is_primary_key = 1
    ) pk
    ON pk.table_name = object_name(a.id)
    AND pk.column_name = a.name
) dt
GROUP BY dt.schema_name, dt.table_name;

```

The following table lists examples of outputs:

Schema name	Table name	Column name	Column order
Person	Address	AddressID	AddressID
Person	AddressType	AddressTypeID	AddressTypeID
Person	BusinessEntity	BusinessEntityID	BusinessEntityID
Person	BusinessEntityAddress	BusinessEntityID, AddressID, AddressTypeID	BusinessEntityID, AddressID, AddressTypeID

Step 2: Generate comparison queries using the metadata results and create SELECT statements

```
SELECT <column_name>
FROM [schema_name].[table_name]
ORDER BY <column_order>;
```

The following is an example of a generated query:

```
SELECT BusinessEntityID, AddressID, AddressTypeID
FROM [Person].[BusinessEntityAddress]
ORDER BY BusinessEntityID, AddressID, AddressTypeID;
```

Step 3: Validation

1. Run the metadata query in both databases.
2. Generate and run SELECT statements for each table.
3. Compare the results between source and target databases:
 - Row counts should match.
 - Data values should be identical.
 - Check for data-type conversion issues.

Step 4: Validate row count

```
SELECT COUNT(1) AS total_rows
```

```
FROM [schema_name].[table_name];
```

Step 5: Update your application's configuration to point to the new database

1. Update the security group.
2. Modify the DNS connection string as needed to connect to the target database.

Testing the migration

The testing process can help you identify overlooked issues during development, such as incorrectly converted queries or missing indexes. And it may reveal the need for database engine tuning or query modifications based on workload performance.

Functional testing, which includes unit tests for application workflows, helps to ensure seamless integration with your new database. Performance testing helps to optimize your database by verifying acceptable response times and identifying bottlenecks.

While both manual and automated testing methods exist, we recommend an automated method because it's more efficient, especially for additional testing cycles.

Phase 4: Operate and optimize

Optimize your database by assessing your application stack for cost, performance, security, and resiliency. For example, use automatic scaling features to add more read replicas during peak hours, and then remove them during off-peak hours when costs are lower.

You can also use AWS services that integrate with Amazon RDS databases, such as directing your database engine logs to Amazon CloudWatch Logs for analysis, and use Performance Insights to help you analyze and visualize your database. For more information, see [Best practices for configuring performance parameters for Amazon RDS for SQL Server](#).

Resources

AWS documentation

- [Authenticate Microsoft SQL Server on Amazon EC2 using AWS Directory Service](#)
- [Enable transparent data encryption in Amazon RDS for SQL Server](#)
- [Preparation and assessment for migrating from SAP ASE to Amazon Aurora MySQL](#)
- [Setting up a network for a replication instance](#)
- [Using a Microsoft SQL Server database as a target for AWS Database Migration Service](#)

AWS services

- [Amazon CloudWatch](#)
- [Amazon RDS for SQL Server](#)
- [Amazon RDS Performance Insights](#)
- [Amazon Simple Notification Service \(Amazon SNS\)](#)
- [Amazon Simple Storage Service \(Amazon S3\)](#)
- [AWS CloudTrail](#)
- [AWS Database Migration Service \(AWS DMS\)](#)
- [AWS Storage Gateway](#)

Blog posts

- [Best practices for configuring performance parameters for Amazon RDS for SQL Server](#)
- [Integrate an SAP ASE database to Amazon S3 using AWS Storage Gateway](#)
- [Migrate your SAP ASE \(Sybase ASE\) database to Amazon RDS for SQL Server](#)
- [Validate database objects after migrating from SAP ASE to Amazon RDS for SQL Server or Microsoft SQL Server](#)

Sybase ASE and SQL Server

- [Best practices for working with SQL Server](#)
- [SAP Adaptive Server Enterprise \(SAP ASE\)](#)
- [Transact-SQL](#)
- [Tabular Data Stream \(TDS\)](#)

Document history

The following table describes significant changes to this guide.

Change	Description	Date
Initial publication	—	July 16, 2025